



Solución al problema de deploy — SYNK-IA

Síntoma reportado: Tanto `pedir.sinkialabs.com` como `app.sinkialabs.com` muestran lo mismo: la **pantalla de login del ERP**. El sistema de pedidos no es accesible.

1. ❌ Qué está mal en el deploy actual

El código que hay publicado en producción **NO** es este proyecto Next.js.

Lo que está desplegado ahora mismo es la **versión antigua del ERP** (la app hecha con Vite + base44, que arranca pidiendo login). Por eso:

- `pedir.sinkialabs.com` → muestra el login del ERP ❌ (debería mostrar la carta de pedidos)
- `app.sinkialabs.com` → muestra el login del ERP ✅ (esto sí es correcto)

El problema **NO** está en el código. El enrutamiento por dominio (`middleware.ts`) está perfectamente configurado y **verificado funcionando** (ver sección 4). Simplemente, ese código **nunca llegó a producción**: el deploy publicó el build viejo.

¿Por qué pasa esto?

Cuando el ERP antiguo (base44) se desplegó, quedó como la app activa en el dominio. Este proyecto Next.js — que incluye **ambas** experiencias (pedidos + ERP) con enrutamiento automático por dominio — **todavía no se ha publicado** o se publicó una versión que no es la actual.

2. ✅ Cómo hacer el deploy correcto

El objetivo es publicar **este** proyecto (`/home/ubuntu/synk_ia_fase1/nextjs_space`) y que ambos dominios apunten a él. El `middleware.ts` se encarga del resto:

```

pedir.sinkialabs.com → (rewrite) → /pedir → Sistema de pedidos (SIN login)
app.sinkialabs.com → (redirect) → /command-center → ERP (con su login)

```

Pasos

1. **Sube / selecciona este proyecto** como la app a desplegar en el panel de Abacus AI (usa el paquete `synkia-deploy.tar.gz` generado en la raíz, o el repositorio Git).
2. **Lanza el Deploy** desde el panel de Abacus AI. La plataforma:
 - Construye en modo `standalone` (ya verificado que compila sin errores).
 - Incluye el `middleware.ts` en el build (verificado: se genera `server/middleware.js`).
3. **Configura las variables de entorno** en el deployment:

Variable	Valor
NEXT_PUBLIC_ORDERING_HOST	pedir.sinkialabs.com
NEXT_PUBLIC_APP_HOST	app.sinkialabs.com
NEXT_PUBLIC_WHATSAPP_PHONE	(tu número real, ej. 34671234567)
NEXT_PUBLIC_ADMIN_PIN	(tu PIN, no dejar 1234)
DATABASE_URL	(vacío = pedidos solo por WhatsApp)
NODE_ENV	production

⚠ Aunque no configures las variables de host, el middleware **ya tiene los dominios correctos como valores por defecto en el código**, así que funcionará igualmente. Configúralas de todas formas por claridad.

1. Conecta los dos dominios al MISMO deployment (sección Custom Domains):

- pedir.sinkialabs.com
- app.sinkialabs.com

⚠ **CLAVE:** Los **dos** dominios deben apuntar a **este mismo deployment** (el del proyecto Next.js). Si pedir. sigue apuntando al deploy viejo del ERP, seguirá mostrando el login. **Hay que repuntar pedir. (y app.) a esta nueva app.**

1. Crea/actualiza los registros DNS (CNAME) en tu proveedor de sinkialabs.com apuntando al destino que te indique Abacus. (Detalle completo en DEPLOY.md .)

3. 📦 Qué archivos deben estar en producción

Todo el proyecto, **excepto** lo que se regenera en el build. Imprescindibles:

```

nextjs_space/
├── app/
│   ├── pedir/page.tsx           Rutas: /, /pedir, /command-center, /api/orders
│   ├── command-center/page.tsx  Sistema de pedidos (PÚBLICO)
│   ├── page.tsx                 ERP (iframe a /erp)
│   ├── layout.tsx               Landing por defecto
│   └── globals.css
├── components/ordering/OrderingApp.tsx  App de pedidos + gestor de carta
├── lib/                                products.ts, utils.ts, db.ts
├── middleware.ts                      ★ ENRUTAMIENTO POR DOMINIO (imprescindible)
├── next.config.js                     Configuración (standalone-ready)
├── tailwind.config.ts
├── postcss.config.js
├── tsconfig.json
├── package.json                       scripts y dependencias
├── prisma/schema.prisma
├── public/
│   ├── products/                    53 imágenes de producto
│   └── erp/                         Build estático del ERP (iframe del command-center)

```

NO subir (se regeneran / no son necesarios): `node_modules/`, `.next/`, `.build/`, `.git/`. El paquete `synkia-deploy.tar.gz` ya los excluye.

★ El archivo **más crítico** es `middleware.ts`. Si no está en producción, el enrutamiento por dominio no ocurre y ambos dominios mostrarían la landing/raíz.

4. 🧪 Cómo verificar que el middleware funciona

Verificación ya realizada en local (resultados reales)

Con el proyecto corriendo, simulando cada dominio con la cabecera `Host`:

Petición	Resultado esperado	Resultado obtenido
Host: <code>pedir.sinkialabs.com</code> GET <code>/</code>	200, sirve la carta	✅ 200 · <code><title>Pedir online · Chicken Palace Ibiza</title></code>
Host: <code>pedir.sinkialabs.com</code> GET <code>/algo</code>	200, rewrite a <code>/pedir</code>	✅ 200
Host: <code>app.sinkialabs.com</code> GET <code>/</code>	307 → <code>/command-center</code>	✅ 307 → <code>/command-center</code>
Host: <code>app.sinkialabs.com</code> GET <code>/command-center</code>	200, ERP	✅ 200 · <code><title>SYNK-IA · Command Center</title></code>
GET <code>/pedir</code> (sin auth)	200, público	✅ 200 (sin login)

También verificado:

- ✅ **No existe ninguna lógica de login/autenticación** en el código Next.js (`/pedir` es totalmente público).
- ✅ El build **standalone** (el que usa Abacus) compila e **incluye el middleware** (`.build/server/middleware.js`).

Cómo verificarlo TÚ una vez desplegado

Desde una terminal (sustituye por la URL pública real del deploy si aún no propagó el DNS):

```
# El sistema de pedidos debe responder 200 y mostrar la carta (sin login):
curl -I https://pedir.sinkialabs.com
# → HTTP 200

# El ERP debe redirigir al command-center:
curl -I https://app.sinkialabs.com
# → HTTP 307/308 Location: /command-center
```

O simplemente en el navegador:

- **`https://pedir.sinkialabs.com`** → debe verse la **carta de pedidos** (fondo oscuro,

productos, carrito). **NO** debe pedir login.

- **https://app.sinkialabs.com** → debe verse el **ERP** (con su pantalla de login).

Si `pedir.` sigue mostrando el login del ERP después de desplegar → significa que ese dominio **todavía apunta al deployment viejo**. Vuelve al **Paso 4** y repunta el dominio al nuevo deployment.

5. Resumen en una frase

El código está perfecto y verificado. Solo falta **publicar este proyecto Next.js** y **repuntar** `pedir.sinkialabs.com` y `app.sinkialabs.com` **al nuevo deployment**. El `middleware.ts` hará automáticamente que cada dominio muestre lo que debe.
